商品展示的业务依据

首屏 1 秒、三层缓存与深分页反模式

gomall · 业务侧 deck

今日大纲

首屏 1 秒决定订单

读路径：Cache Aside 是怎么挡住 DB 的

写路径：双删的业务必要性

HTTP cache：把流量挡在源站之外

三态故障：击穿 / 穿透 / 雪崩

反例：product/list 全表 COUNT

业务码、客诉与监控口径

首屏 1 秒决定订单

这一节回答四个问题

- 商品详情页是首屏黄金 1 秒，缓存命中 vs 回源 DB 在业务上差多少。
- Cache Aside 读路径 + 双删写路径，为什么**业务必须接受**最终一致而不是强一致。
- HTTP cache (ETag + Cache-Control) 哪几个接口能挂、TTL 怎么定。
- `product/list` 跑 24.5 RPS / p95 2.5s 是怎么从业务侧解读为反例的。

对应代码 *routes/router.go:39–45 / middleware/httpcache.go / repository/cache/product.go / service/product.go*

首屏延迟与转化率：业务侧的硬性约束

首屏延迟	跳出率上升	业内常见处理
$\leq$ 1s	基线	不动
1 – 3s	+30%	进入优化排期
3 – 5s	+90%	列入 P0 故障复盘
$\geq$ 5s	用户基本走完	直接告警呼叫

Akamai 和 Google 在 2017 / 2018 公开的转化率曲线，是后端做缓存预算的业务依据。

gomall 的 `product/show` 压测 p95 = 3.01ms，留出 300 倍余量给业务侧的图片加载、首屏渲染。*stressTest/REPORT.md § 各链路压测结果（行 product/show）*

命中缓存 vs 回源 DB 的业务收益

路径	单次耗时	956K 行表能承担 QPS	业务侧含义
浏览器 304	~ 30ms (RTT)	不打到源站	流量被 CDN/浏览器卸掉
Redis 命中	~ 1ms	数万级	单机 Redis 上限
PK 回源 DB	3.01ms (p95)	6 万级	与 ping 同量级
全表扫	2.5s (p95)	24.5	直接告警

回源”不可怕，”扫表”才可怕。gomall `/product/show` 走主键 + 缓存兜底，业务侧从来不是瓶颈；`/product/list` 全表 COUNT 才是要修的那个。*stressTest/REPORT.md §1 基线 vs 业务接口；§3 商品列表反模式*

读路径：Cache Aside 是怎么挡住 DB 的

Cache Aside 读路径

- 读：先读缓存；缺则回源，再写回缓存。
- gomall 实现里多一道 `SETNX` 回源锁：缓存击穿时 **只放一个请求**打 DB，其它请求等 50ms 再读一次。
- 业务结果：商品详情压测 62K RPS，与 `/ping` 基线几乎重合。

service/product.go:42–68 ProductShow 三段式

读路径关键代码

Listing 1: *service/product.go:42–68 ProductShow*

```go
cached := &types.ProductResp{}
if err := cache.GetProductDetail(ctx, req.ID, cached); err == nil {
    return cached, nil // L1: 命中, ~1ms
}
locked, _ := cache.TryProductLock(ctx, req.ID) // L2: SETNX 抢回源权
if !locked {
    time.Sleep(50 * time.Millisecond) // L2': 让锁持有者回填
    if err := cache.GetProductDetail(ctx, req.ID, cached); err == nil {
        return cached, nil
    }
} else { defer cache.UnlockProduct(ctx, req.ID) }
pResp, err := s.loadProductFromDB(ctx, req.ID) // L3: 锁持有者打 DB
if err != nil { return nil, err }
_ = cache.SetProductDetail(ctx, req.ID, pResp) // L4: 回填, TTL=10min
return pResp, nil
```

读路径四步的业务含义

L1 命中 99% 的请求走这里。Redis ~ 1ms，对业务侧等价于” 没有数据库”。

L2 SETNX 缓存失效/被双删的瞬间，防止”100 个并发同时打 DB” 导致雪崩，业务侧叫**缓存击穿**。

L2’ 等 没抢到锁的请求 sleep 50ms 再读一次。读到就走 L1；读不到再硬回源（兜底，不能因为锁丢请求）。

L4 回填 TTL = `ProductDetailTTL = 10min`，这是 Redis 层；HTTP 层是 60s。两层 TTL 不一样，下一节解释。

*repository/cache/product.go:14 ProductDetailTTL = 10*time.Minute*

写路径：双删的业务必要性

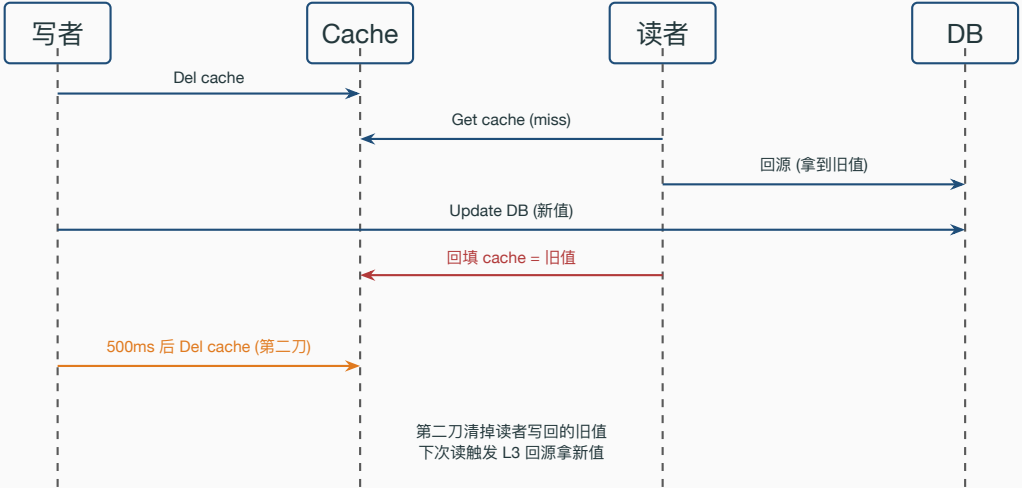
为什么不能” 写库 + 写缓存”

- 直觉做法：商家改价 → `UPDATE product` → `SET cache`。
- 业务现实里两个并发请求会：
  1. T1 改价 100 → 99
  2. T2 改价 100 → 88
  3. T2 先到 DB（写 88），T1 后到 DB（覆盖为 99）
  4. T1 后写 cache (99)，T2 先写 cache (88)
  5. DB = 99，Cache = 88 ——**永久不一致**
- 客诉版本：” 为什么列表写 88，详情写 99？ ”

> 业务侧不能依赖” 写缓存”，只能依赖” 删缓存”。

典型双写不一致问题；详见 *Facebook TAO* 论文 *§4*

延迟双删：第二刀挡哪个窗口



repository/cache/product.go:64–74 DoubleDeleteAsync (interval=500ms)

写路径关键代码：删-写-延迟删

Listing 2: *service/product.go:271–292 ProductUpdate*

```go
product := &model.Product{ /* 价格/标题等字段 */ }
_ = cache.DelProductDetail(ctx, req.ID) // 第一刀: 写库前先删
err = dao.NewProductDao(ctx).UpdateProduct(req.ID, product)
if err != nil {
    log.LogrusObj.Error(err); return
}
cache.DoubleDeleteAsync(req.ID, 0) // 第二刀: 500ms 后再删
emitProductChanged(ctx, req.ID, "update") // outbox -> RMQ -> ES
```

业务侧约定：商家改价后**最长 500ms + Redis TTL** 才能保证不出旧值；详情页 TTL=10min 时窗口实际由双删的 500ms 兜住。

业务侧为什么接受”500ms 旧值”

- 强一致方案（2PC / 同步写）单次写要 50–200ms，且任何下游慢都会让” 改价” 按钮卡住。
- 客服话术：” 改价后 1 秒内可能仍显示老价格，刷新即可。”——这是和产品/运营事先谈好的口径，不是 bug。
- 反过来，如果业务允许 1s 误差，第二刀的 500ms 已经是过设计；改成 200ms 也行。
- 真正不能容忍旧价的场景是**下单页**：那里 `/orders/create` 必须从 DB 现读价格，不走 cache。

> Cache Aside 是业务 SLA ” 最终一致 + $\leq$ 1s 窗口” 的工程实现，不是教科书最优解。

service/product.go:271–292; repository/cache/product.go:64–74

HTTP cache：把流量挡在源站之外

四层缓存漏斗

浏览器 cache (Cache-Control max-age) — >60% 流量在此截断

CDN / 反向代理 (ETag + If-None-Match) — 再截 15–20%

Redis (product:detail:{id}) — 再截 99% 剩余

MySQL (product PK) — 兜底回源

业务侧的数字：上一档 cache 卸下的流量，比下一档便宜一个数量级。浏览器 304 没有源站成本，CDN 一次回源覆盖一千个用户，Redis 一次回填覆盖 10min 内所有同商品请求。*routes/router.go:39–45* 四档挂载点

每层命中率怎么算

层	命中率定义	度量口径
浏览器	`304 / (200+304)`（按 referer 维度）	前端埋点 `performance.getEntriesByType`
CDN	`(edge_hit + 304) / total_req`	CDN 厂商日志 `cache_status=HIT`
Redis	`1 - cache_miss / total_req`	`INFO stats` 中 `keyspace_hits / (hits+m`
DB	反向口径：`1 -` 上三层总命中率	由 prometheus `db_qps / req_qps` 反推

业务侧不会单独看 Redis 命中率，会看**端到端漏斗**：浏览器 60%、CDN 在剩下 40% 里再截 20%（绝对值 8%）、Redis 在剩下 32% 里再截 99%（绝对值 31.7%），DB 最终承担 **0.32%** 的原始流量。10k QPS 进来，DB 实际只看到 32 QPS。*Prometheus* 指标：

`redis_hits_total / http_requests_total{code="304"}`

命中率链式公式与业务含义

设各层命中率为 h_1, h_2, h_3，DB 流量 $= Q \cdot (1-h_1)(1-h_2)(1-h_3)$。

- **乘性效应**：任一层命中率掉 10%，DB 流量翻倍不止。Redis 99% $\rightarrow$ 90% 时 DB 流量从 0.32% 涨到 3.2%，单机 MySQL 直接吃不消。
- **业务对齐**：浏览器层 60% 不是技术指标，是**产品约定 max-age=60s** 换来的。产品坚持”改价 1s 内必须可见”时这层掉到 0。
- **容量规划反推**：先定 DB 能扛多少 QPS，倒推上游每层最低命中率，再倒推 TTL。

结合 stressTest/REPORT.md DB 单点 6 万级 QPS 与 product/show 62K RPS 的对齐

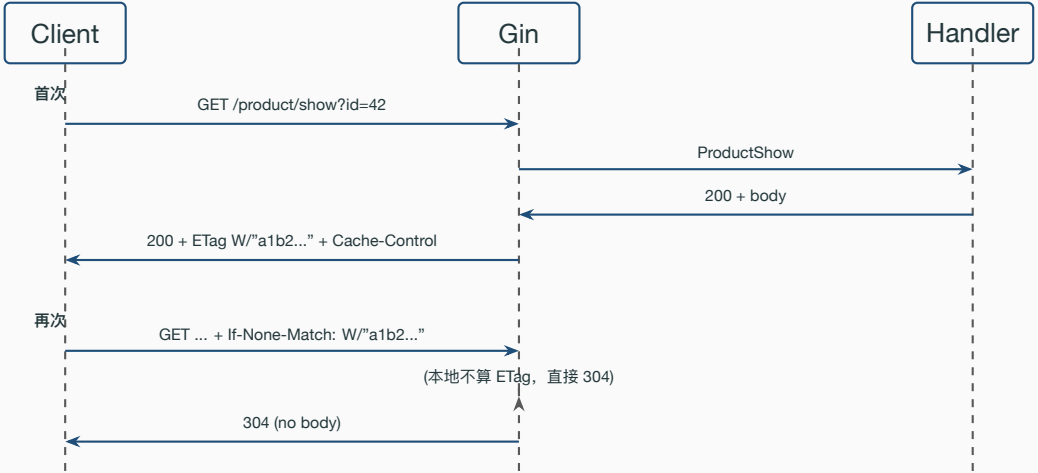
CDN 选型与命中率掉档的运维剧本

方案	节点 / 成本	命中率 KPI	适用
阿里云 CDN	国内 2800+ / 0.2 元/GB	95%+	国内电商主流
Cloudflare	全球 300+ / 0.05 USD/GB	92%+	出海 / 多区域
自建 ATS	0 边际，前期 1M+	看运维	月流量 >10 PB
回源直发	0 + 源站带宽费	N/A	试点 / 内网（gomall 当前）

掉档剧本：浏览器突降 = 前端发版改了 URL query，回滚或加 `Vary` 白名单；CDN 突降 = 回源带 cookie，`/product/show` 必须无 session；Redis 突降 = `evicted_keys` 在涨，扩容或调 `allkeys-lfu`；DB QPS 突涨但 Redis 没变 = 新接口漏挂 cache，grep `HTTPCache` 白名单。

Cloudflare cache analytics doc / 阿里云 CDN 计费规则 / Prometheus alert `cache_hit_ratio < 0.95 for 5m`

ETag 304 流程



middleware/httpcache.go:53–99 ETag 比对与 304 分支

ETag 中间件关键代码

Listing 3: *middleware/httpcache.go:78–99 比对 + 304*

```go
etag := weakETag(buf.body.Bytes()) // SHA-256 前 16 字节 -> W/"..."

if c.Request.Header.Get("If-None-Match") == etag {
    h := original.Header()
    h.Set("ETag", etag); h.Set("Cache-Control", cacheControl)
    h.Del("Content-Length"); h.Del("Content-Type")
    original.WriteHeader(http.StatusNotModified) // 304: 不写 body
    return
}
h := original.Header()
h.Set("ETag", etag); h.Set("Cache-Control", cacheControl)
h.Set("Content-Length", strconv.Itoa(buf.body.Len()))
original.WriteHeader(http.StatusOK)
_, _ = original.Write(buf.body.Bytes()) // 200: 正常回写
```

ETag 中间件逐行讲解 + 为什么 POST 不挂

- **弱 ETag (`W/"..."`)**：只承诺” 语义等价”，不承诺字节级等价。压缩/分块都不影响命中，业务上更适合 JSON。
- **先 `c.Next()` 再比对**：handler 算完才能拿 body，中间件用 `cacheBuffer` 包 `ResponseWriter`，代价是多一份内存。
- **只对 `status=200` 生效**：4xx / 5xx 直接透传，业务码 70001 (限流) 不会被错误地长期缓存。
- **304 后删 `Content-Length` / `Content-Type`**：标准要求，否则上游 nginx / CDN 把空 body 算成 0 字节但仍发头，浏览器会拒收。
- **POST/PUT 不挂**：写接口挂 cache 等于” 重复触发同一动作”，与幂等冲突；`POST /orders/create` 命中 cache 会让用户以为下单成功但实际没打到后端，对账少一笔。

middleware/httpcache.go:47–99 + 方法判断:54–58

TTL 怎么定：业务节奏决定

接口	max-age	业务理由	风险
`/product/show`	**60s**	单品价格/库存可能改	改价 1min 内显示老价
`/product/list`	30s	列表翻页频繁，改卖家少	上架 30s 内不可见
`/category/list`	**300s**	类目几乎不变	几乎无
`/carousels`	300s	运营每天换一次轮播图	几乎无

数字不是抄的，是和**产品同学**对齐过的” 用户能容忍多久看到旧值”。

60s 是为了和 ProductDetailTTL (10min) 解耦：即使 Redis 还是旧值，HTTP 层 60s 后会强制重新协商一次。*routes/router.go:39 (30s) / :40 (60s) / :44 (300s) / :45 (300s)*

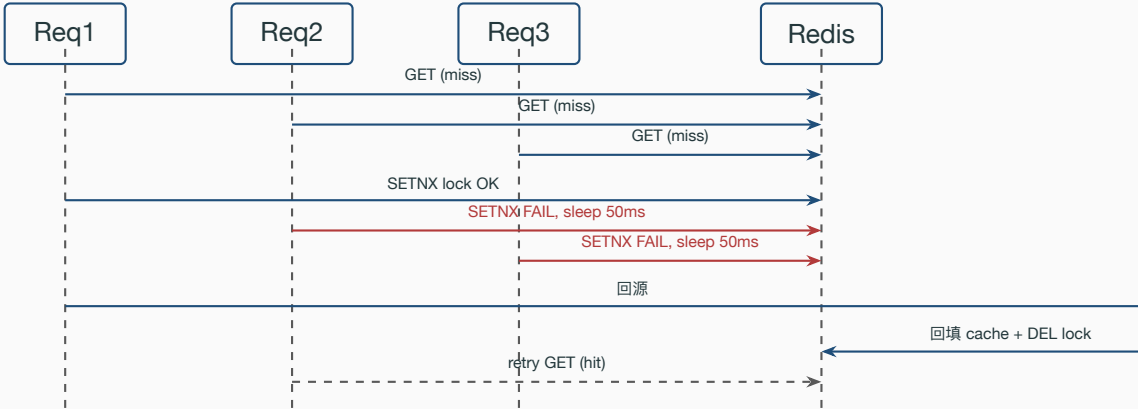
三态故障：击穿 / 穿透 / 雪崩

三种缓存故障的区别

名称	触发条件	后果	
击穿	单个热 key 过期，瞬间 N 个请求回源	单点 DB 被打爆	三态共
穿透	请求查**不存在**的 key，cache 永远 miss	每次都打 DB，CPU 100%	
雪崩	大量 key 同一时刻过期	整片回源，DB 链接池打满	

同点：cache miss 引起 DB 流量**阶跃**上涨。区别在 **key 的范围**（单 key / 不存在的 key / 多 key）和**时间分布**（瞬时 / 持续 / 同步）。对应防护手段三种：**SETNX 单飞 / 布隆过滤器 / TTL 抖动**。下面分别讲。对应 *service/product.go:50–58 SETNX* + 后续准备引入的布隆过滤器和 *TTL jitter*

击穿：SETNX 单飞回源



service/product.go:50–58 + repository/cache/product.go:56–62

穿透：布隆过滤器挡住” 不存在” 的 id

- 攻击场景：脚本爬虫扫 `/product/show?id=99999999`，DB 返回 `record not found`，cache 不会回填 nil。下一次请求继续打 DB。
- 直觉修法 1：cache 回填 `nil` 标记 + TTL 60s。代价：脏数据需要等过期，正常 id 上架后 60s 不可见。
- 直觉修法 2：DB 前置布隆过滤器，启动时灌入所有合法 `product_id`，新建商品时增量 `BF.ADD`。99.99% 不存在的 id 在 Redis 层直接拒绝。
- gomall 当前没接，但已经预留接口：`cache.ProductExists(id)` 在 ProductShow 入口可加一行短路。

规划中：*repository/cache/bloom.go (Redis 4.0+ `BF.RESERVE product:bf`)*

布隆过滤器伪代码

Listing 4: 规划：*repository/cache/bloom.go BF.MEXISTS* 用法

```go
// ProductExists 用布隆过滤器快速判 id 是否可能存在
// 误判率 0.01%; 误判方向只可能是 false-positive (说存在但实际不存在)
// 不会出现 false-negative (说不存在但实际存在), 业务侧零脏读
func ProductExists(ctx context.Context, id uint) (bool, error) {
    res, err := RedisClient.Do(ctx, "BF.EXISTS", "product:bf", id).Bool()
    if err != nil { return true, err } // BF 故障时放行, 走原回源链路
    return res, nil
}

// ProductShow 入口处加短路
exists, _ := cache.ProductExists(ctx, req.ID)
if !exists {
    return nil, e.ErrProductNotFound // 0.01ms 直接拒绝
}
```

雪崩：TTL 抖动与多级兜底

- 触发场景：商家批量上架 1000 个 SKU，全部用默认 `ProductDetailTTL = 10min`。10 分钟后**同一秒**集中过期，回源瞬时打 1000 次 DB。
- 修法 1：TTL **加随机抖动**。写入时 `ttl = base + rand(0, 0.2*base)`，把过期时间打散到 2 分钟窗口。
- 修法 2：**多级 TTL**。L1 cache 10min、L2 cache 1h，L1 过期回 L2，L2 过期回 DB。gomall 暂未启用 L2，路线图上是 Redis cluster 上的 `ro-replica`。
- 修法 3：**熔断兜底**。DB 拒连时整链路降级返回上一版 cache（哪怕过期），客服话术接受”价格延迟 1 分钟”。

规划：*repository/cache/product.go SetProductDetail* 加 *jitter* 实现

TTL 抖动落地

Listing 5: 规划改造：*SetProductDetail* 加 *jitter*

```go
// SetProductDetail 写缓存，带 0-20% 随机抖动避免批量同时过期
func SetProductDetail(ctx context.Context, id uint, val interface{}) error {
    b, err := json.Marshal(val)
    if err != nil {
        return err
    }
    // jitter: 10min * (1.0 + rand[0, 0.2]) -> 10-12min 区间均匀分布
    jitter := time.Duration(rand.Int63n(int64(ProductDetailTTL / 5)))
    return RedisClient.Set(ctx, ProductDetailKey(id), b, ProductDetailTTL+jitter).Err()
}
```

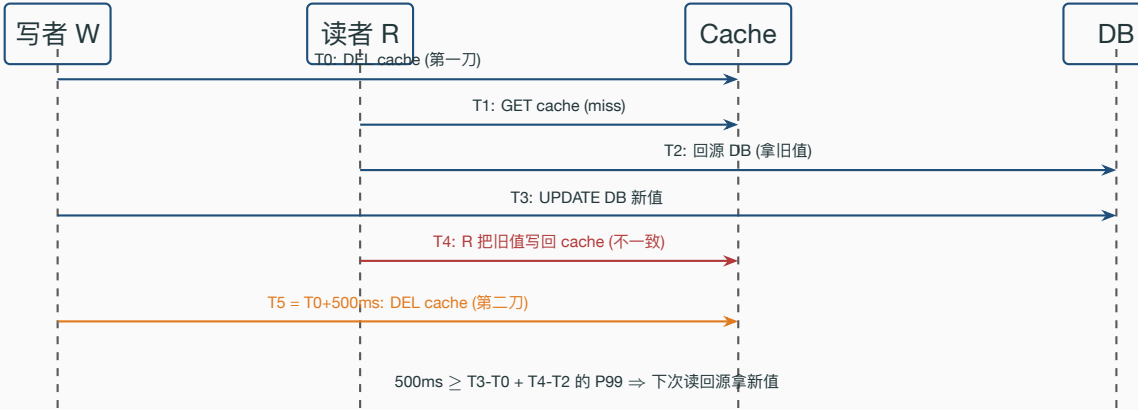
数学上：1000 个 key 集中写入，原方案在 `T+10min` 那 1 秒里有 1000 个 key 过期；加 20% 抖动后均匀分布到 120 秒，每秒平均 8 个过期，DB 瞬时负载降低 **120 倍**。

第二刀间隔的取舍

第二刀间隔	覆盖率	副作用
100ms	覆盖 ~ 80% 回填窗口	改价后体感最快，但漏窗仍有
500ms	覆盖 ~ 99% 回填窗口	gomall 当前值，平衡点
1s	覆盖 ~ 99.9% 回填窗口	改价后用户最长等 1s + TTL
3s	几乎全覆盖	用户体感差，且改价高峰时 goroutine 堆积

数字怎么来：取决于**读路径回填耗时**的 P99。gomall 单次回填 = DB 查询 (3ms) + JSON marshal (1ms) + Redis SET (1ms) ≈ 5ms。考虑 GC / 抖动 / 网络重传，P99 不超过 100ms；500ms 留 5 倍余量。*repository/cache/product.go:16 ProductDelayInterval = 500ms*

并发场景下的双删时序



推导：*T4-T0* = 写库耗时 + 读者从 *miss* 到回填耗时

Redis cluster 上的热 key 怎么找

- 现象：单分片 CPU 100%、网络出带宽打满，其他分片闲；典型表现是**秒杀商品 / 双 11 主推品**的 `product:detail:{id}`。
- 探测方法 1：`redis-cli --hotkeys`（基于 LFU 采样，会暂停 4 秒，仅离线用）。
- 探测方法 2：开 `slowlog` + `MONITOR` 抽样 1% 流量，按 key 聚合统计 QPS。
- 探测方法 3：在 Gin 层埋点，记录每个 `product_id` 的请求计数到 `sync.Map`，1 分钟 `dump` 一次到 Prometheus；超过阈值（如 >1000 QPS）告警。
- gomall 当前是方法 3，路线图上接 Redis cluster 后会切到方法 2 + 客户端旁路缓存。

规划：*middleware/hotkey.go + Prometheus counter `product_view_total`*

热 key 三种拆分策略

本地 process cache 在 Go 进程内加 `LRU(size=100, ttl=5s)`，对极热 key 直接走本地，绕过 Redis。代价是**一致性窗口扩到 5s**，仅适用于”商品详情”这类对一致性要求低的字段。

Key 分片 把 `product:detail:42` 拆成 `product:detail:42:shard0..N`，读时随机选一片，写时全部更新。N 取 10 时单分片 QPS 降 10 倍，代价是写放大。

读写分离 热 key 写 master、读从多副本 `ro-replica` 上读。最终一致窗口 = master-replica 同步延迟（约 10ms）。

业务侧选哪个看**读写比例**。商品详情读写 >1000:1，本地 cache 最划算；秒杀库存读写 1:1，必须分片。方案对比：*Twitter Manhattan / Facebook MCRouter* 公开论文

反例：product/list 全表 COUNT

反例数字：24.5 RPS / p95 2.5s

接口	实测 RPS	p95
`/ping` 基线	64,254	3.51ms
`/product/show`	62,226	3.01ms
`/product/list` (反例)	**24.5**	**2.50s**

同样是商品域的 GET 接口，吞吐差 **2,500 倍**，p95 差 **800 倍**。

区别只有一个：`ShowProduct` 走主键，`ListProducts` 多了一个 `SELECT COUNT(*)`。

stressTest/REPORT.md § 各链路压测结果（行 product/list）

扫表示意：1M 行的 COUNT 怎么炸的

```
SELECT COUNT(*) FROM product WHERE category_id=?
```

全表扫: 每次请求 ~ 2 s

50 VU * 2s → 24.5 RPS

product 表 956K 行 / 364 MB

- `Count(&total)` 在没有 `category_id` 索引时退化为全表扫。
- 即使有索引，InnoDB 的 `COUNT(*)` 不能像 MyISAM 那样常量返回，仍要走索引页。
- 业务侧问的从来不是” 商品一共多少件”，而是” 我这一页有多少件”——total 大多是**产品需求过度**。

repository/db/dao/product.go:58–64 CountProductByCondition

反例代码

Listing 6: *repository/db/dao/product.go:58–64*

```go
// CountProductByCondition 根据情况获取商品的数量
func (dao *ProductDao) CountProductByCondition(condition map[string]interface{}) (total int64,
     err error) {
   err = dao.DB.Model(&model.Product{}).
      Where(condition).Count(&total).Error // 1M 行表 + 无 category_id 索引 -> 全表扫
   return
}
```

service 调用点： *service/product.go:195–204 ListProducts 同时调 ListProductByCondition + CountProductByCondition*

三种修法对业务的代价

方案	代价	业务可见变化
建 `category_id` 索引	写入慢 5–10%	几乎无
近似计数 (`TABLE STATUS`)	total 有 5% 误差	显示” 约 1,000 件”
物化视图 (`category_count`)	异步刷新延迟	数字偶有 5min 滞后
干掉 total，前端无限滚动	产品改交互	翻页器消失

后端常常想直

接选” 建索引”，但和产品/运营对齐后会发现**无限滚动**才是真正合理的选择：用户分页到第 50 页的概率 $< 0.1\%$，业务上没人在乎 total。*stressTest/REPORT.md §3* 商品列表反模式改进方向

近似计数：用 INFORMATION_SCHEMA 替代 COUNT(*)

Listing 7: 规划改造：*repository/db/dao/product.go ApproxCount*

```go
// ApproxCount 近似计数: 从 INFORMATION_SCHEMA 拿 InnoDB 估算行数
// 误差 5-10%; 1M 行表从 2s 降到 1ms, 4 个量级
func (dao *ProductDao) ApproxCount() (int64, error) {
    var rows int64
    err := dao.DB.Raw(`SELECT TABLE_ROWS FROM INFORMATION_SCHEMA.TABLES
                    WHERE TABLE_SCHEMA = DATABASE()
                      AND TABLE_NAME = 'product'`).Scan(&rows).Error
    return rows, err
}
```

`TABLE_ROWS` 是 InnoDB 统计样本估算值。按 category 分组要走**物化视图**：另起 `product_count_by_cat` 表，binlog/outbox 异步增减。

物化视图 / 无限滚动：选哪个

方案	一致性	复杂度	查询耗时
`(category, on_sale)` 复合索引	强	加索引一行	50–200ms
物化视图 `product_count_by_cat`	5min 最终一致	中（需 outbox）	<1ms
近似计数 `TABLE_ROWS`	弱（10% 误差）	极低	<1ms
无限滚动（cursor）	不需要 total	中（前端改）	<1ms

物化视图临界点：**QPS >1k && 类目稳定**。gomall 当前 24.5 RPS 没到，先建索引。

无限滚动是最优解：用户翻第 50 页概率 <0.1%，total 价值极低。`order/list` 已用游标 + 缓存把 QPS 从 8.3 拉到 **58,406**（~7000 倍），商品列表可复用同样模板：cursor = `(sort_key, id)` 二元组。游标分页模板已在 *stressTest/REPORT.md §2* 验证

容量规划：1k / 10k / 100k QPS 链路资源估算

入口 QPS	1,000	10,000	100,000
浏览器卸载（60%）	400 到源	4,000 到源	40,000 到源
CDN 卸载（20% 再截）	320 到源站	3,200 到源站	32,000 到源站
源站 Redis（99% 命中）	3.2 到 DB	32 到 DB	320 到 DB
DB 实际负载	<1% 单机	10% 单机	接近上限
源站 CPU（按 50K RPS/核）	0.01 核	0.1 核	1 核

100k

QPS 才把 DB 推到 320 QPS，远低于单机 6 万级 PK 上限。**单机够用**，所以 gomall 在百万级 DAU 之前不需要分库分表，只需要把 cache 命中率守住 99%。基线数字来自 *stressTest/REPORT.md /ping 64K RPS / product/show 62K RPS*

业务码、客诉与监控口径

业务码：60002 的真实含义

状态码	含义	客服话术
60001	幂等 token 无效	” 请刷新页面后重试”
60002	幂等正在处理	” 您的请求正在处理，1 秒后自动刷新”
70001	限流命中	” 活动太火爆，请稍后重试”
70002	熔断开启	” 支付服务暂忙，1 分钟后”

注意 60002 不是

cache 命中（HTTP 304 才是），而是**幂等命中**——同一 Idempotency-Key 在第一笔还没结束就再来一次。两者对客服来说都是” 无需重复操作”，话术接近；但监控口径不同：304 不写日志、不计错误率；60002 要计入” 幂等保护次数”。*pkg/e/code.go:53–58* 业务码定义

客诉拆解 + 业务侧最终目标

” 为什么显示老价格” 三种场景：

- **A** 改价后 $\leq$ 500ms：双删第二刀还没到，让用户再刷一次。
- **B** 改价后 1–60s：浏览器/CDN 强 cache，教硬刷新 (`Cmd+Shift+R`) 或运营” 强制下架重新上架”。
- **C** 改价后 $>$ 10min 仍旧值：双删失败或 outbox 没投递，开 P2 工单查 `outbox` 表。

业务侧最终目标：首屏 1s（浏览器 304 + CDN）/ 命中率 >99%（Redis + SETNX）/ 改价 1s 内可见（双删 + outbox）/ 翻页不炸（cursor + 无限滚动）。

> 商品展示不是” 把数据从 DB 取出来”，是**把流量挡在 DB 之外**。

service/product.go:282–289 双删 + emitProductChanged 三联

面试 Q&A（一）

Q1：双删第二刀的间隔为什么是 500ms，不是 200ms 或 1s?
A：业务承诺” 改价 1s 内可见”。第一刀挡 99%，第二刀兜并发读 + 回写窗口；500ms 是”读者 cache miss 到回填完”P99 上界（实测 <100ms）的 5 倍。

Q2：`/product/show` 已 62K RPS 了，为什么还要挂 HTTP cache?
A：62K RPS 是**单机源站极限**，不可水平扩。挂 HTTP cache 后浏览器 / CDN 卸掉 60–80% 流量，源站压力降到 1 万级，留出余量给写路径。

Q3：商家改价后客诉” 显示老价格”，怎么定位?
A：看时间窗口。≤ 1s 是双删窗口（正常）；1–60s 是 HTTP cache（教硬刷新）；> 10min 是 outbox / 双删失败（开工单）。

Q4：为什么 `product/list` 不挂 Redis cache 而 `product/show` 挂?
A：列表 key 维度（`category, page, filter`）笛卡尔积，命中率低；单品按 id 命中率 → 99%。列表更适合 HTTP cache 或前端无限滚动。

面试 Q&A（二）

Q5：弱 ETag 和强 ETag 怎么选？
A：JSON 接口选弱（`W/"..."`）允许压缩/格式变化；二进制资源选强。gomall 全部 JSON，统一弱 ETag。

Q6：商家秒级改价、用户必须秒级看到，怎么办？
A：改业务约定，不改技术。要么 max-age=0 取消强缓存，要么走**下单页直读 DB** 兜底。”既要强缓存又要强一致” 在分布式里没有解。

Q7：缓存击穿、穿透、雪崩三个名词怎么区分？
A：击穿 = 单 key 过期瞬时并发回源（SETNX 单飞）；穿透 = 查不存在的 key 一直 miss（布隆过滤器）；雪崩 = 多 key 同时过期（TTL 抖动 + 多级 cache）。

Q8：单 key 热点（双 11 主推品）打爆 Redis 单分片怎么办？
A：三选一：(1) Go 进程内 LRU 加 5s TTL；(2) key 拆 N 份读时随机选；(3) 走从副本读写分离。读写比 >1000:1 选本地；均衡选分片。

面试 Q&A（三）

Q9：DB 实际承担多少 QPS 怎么算？
A：链式公式 $Q \cdot (1-h_1)(1-h_2)(1-h_3)$。浏览器 60% / CDN 20% / Redis 99%，10k QPS 进 DB 看到 32 QPS。任一层命中率掉 10%，DB 流量翻倍以上。

Q10：监控该看总命中率还是分层命中率？
A：必须分层。Redis 雪崩时浏览器/CDN 没异常，总命中率仍 80%+ 看着像没事。每层独立报警阈值（如 `redis_hit <95% for 5m`）。

Q11：`COUNT(*)` 四种修法业务上怎么选？
A：QPS <1k 加索引；要” 约 1000 件” 文案用近似计数；类目稳定且 QPS >1k 上物化视图；产品愿意改交互直接无限滚动（最优解）。

Q12：百万级 DAU 时需要分库分表吗？
A：不需要。100k QPS 入口、99% 命中率，DB 实际看 320 QPS，远低于 6 万级 PK 上限。先把 cache 命中率守住，再考虑分片。

代码位置一览

读路径 Cache Aside *service/product.go:42–68 ProductShow*

写路径双删 *service/product.go:271–292 ProductUpdate*

Cache 包装 *repository/cache/product.go:30–74 Get/Set/Del/DoubleDeleteAsync*

SETNX 回源锁 *repository/cache/product.go:55–62 TryProductLock*

Double Delete 异步 *repository/cache/product.go:64–74 DoubleDeleteAsync 500ms*

HTTP cache 中间件 *middleware/httpcache.go:47–106 HTTPCache + weakETag*

ETag 304 比对 *middleware/httpcache.go:78–99 If-None-Match 分支*

路由挂载 *routes/router.go:39–40 (show 60s / list 30s) + :44–45 (300s)*

反例：全表 COUNT *repository/db/dao/product.go:58–64 CountProductByCondition*

规划：布隆过滤器 规划 *repository/cache/bloom.go BF.EXISTS*

规划：TTL jitter / 热 key 探测 规划 *SetProductDetail + rand jitter / middleware/hotkey.go*

业务码 *pkg/e/code.go:53–58 (60001/60002/70001/70002)*

压测数字 *stressTest/REPORT.md* 链路压测表 + *§3* 反模式 + 基线 *64,254 / 62,226 / 24.5 RPS*